4. What is the effect on the solution if you change the learning rate and the number of epochs? Does it keep on improving?
5. How does the solution change as you vary the amount of data generated?
    1. Plot the estimation error for $\hat{\mathbf{w}} - \mathbf{w}$ and $\hat{b} - b$ as a function of the amount of data. Hint: increase the amount of data logarithmically rather than linearly, i.e., 5, 10, 20, 50, …, 10,000 rather than 1000, 2000, …, 10,000.
    2. Why is the suggestion in the hint appropriate?

Discussions[80].



3.6 Generalization

Consider two college students diligently preparing for their final exam. Commonly, this preparation will consist of practicing and testing their abilities by taking exams administered in previous years. Nonetheless, doing well on past exams is no guarantee that they will excel when it matters. For instance, imagine a student, Extraordinary Ellie, whose preparation consisted entirely of memorizing the answers to previous years' exam questions. Even if Ellie were endowed with an extraordinary memory, and thus could perfectly recall the answer to any *previously seen* question, she might nevertheless freeze when faced with a new (*previously unseen*) question. By comparison, imagine another student, Inductive Irene, with comparably poor memorization skills, but a knack for picking up patterns. Note that if the exam truly consisted of recycled questions from a previous year, Ellie would handily outperform Irene. Even if Irene's inferred patterns yielded 90% accurate predictions, they could never compete with Ellie's 100% recall. However, even if the exam consisted entirely of fresh questions, Irene might maintain her 90% average.

As machine learning scientists, our goal is to discover *patterns*. But how can we be sure that we have truly discovered a *general* pattern and not simply memorized our data? Most of the time, our predictions are only useful if our model discovers such a pattern. We do not want to predict yesterday's stock prices, but tomorrow's. We do not need to recognize already diagnosed diseases for previously seen patients, but rather previously undiagnosed ailments in previously unseen patients. This problem—how to discover patterns that *generalize*—is the fundamental problem of machine learning, and arguably of all of statistics. We might cast this problem as just one slice of a far grander question that engulfs all of science: when are we ever justified in making the leap from particular observations to more general statements?

In real life, we must fit our models using a finite collection of data. The typical scales of that data vary wildly across domains. For many important medical problems, we can only access a few thousand data points. When studying rare diseases, we might be lucky to access hundreds. By contrast, the largest public datasets consisting of labeled photographs, e.g., ImageNet (Deng *et al.*, 2009), contain millions of images. And some unlabeled image

collections such as the Flickr YFC100M dataset can be even larger, containing over 100 million images (Thomee *et al.*, 2016). However, even at this extreme scale, the number of available data points remains infinitesimally small compared to the space of all possible images at a megapixel resolution. Whenever we work with finite samples, we must keep in mind the risk that we might fit our training data, only to discover that we failed to discover a generalizable pattern.

The phenomenon of fitting closer to our training data than to the underlying distribution is called *overfitting*, and techniques for combatting overfitting are often called *regularization* methods. While it is no substitute for a proper introduction to statistical learning theory (see Boucheron *et al.* (2005), Vapnik (1998)), we will give you just enough intuition to get going. We will revisit generalization in many chapters throughout the book, exploring both what is known about the principles underlying generalization in various models, and also heuristic techniques that have been found (empirically) to yield improved generalization on tasks of practical interest.

3.6.1 Training Error and Generalization Error

In the standard supervised learning setting, we assume that the training data and the test data are drawn *independently* from *identical* distributions. This is commonly called the *IID assumption*. While this assumption is strong, it is worth noting that, absent any such assumption, we would be dead in the water. Why should we believe that training data sampled from distribution $P(X, Y)$ should tell us how to make predictions on test data generated by a *different distribution* $Q(X, Y)$? Making such leaps turns out to require strong assumptions about how P and Q are related. Later on we will discuss some assumptions that allow for shifts in distribution but first we need to understand the IID case, where $P(\cdot) = Q(\cdot)$.

To begin with, we need to differentiate between the *training error* R_{emp}, which is a *statistic* calculated on the training dataset, and the *generalization error* R, which is an *expectation* taken with respect to the underlying distribution. You can think of the generalization error as what you would see if you applied your model to an infinite stream of additional data examples drawn from the same underlying data distribution. Formally the training error is expressed as a *sum* (with the same notation as Section 3.1):

$$R_{\text{emp}}[\mathbf{X}, \mathbf{y}, f] = \frac{1}{n} \sum_{i=1}^{n} l(\mathbf{x}^{(i)}, y^{(i)}, f(\mathbf{x}^{(i)})), \tag{3.6.1}$$

while the generalization error is expressed as an integral:

$$R[p, f] = E_{(\mathbf{x},y)\sim P}[l(\mathbf{x}, y, f(\mathbf{x}))] = \int \int l(\mathbf{x}, y, f(\mathbf{x})) p(\mathbf{x}, y) \, d\mathbf{x} dy. \tag{3.6.2}$$

Problematically, we can never calculate the generalization error R exactly. Nobody ever tells us the precise form of the density function $p(\mathbf{x}, y)$. Moreover, we cannot sample an infinite stream of data points. Thus, in practice, we must *estimate* the generalization error by applying our model to an independent test set constituted of a random selection of examples $\mathbf{X}'$ and labels $\mathbf{y}'$ that were withheld from our training set. This consists of

applying the same formula that was used for calculating the empirical training error but to a test set $\mathbf{X}', \mathbf{y}'$.

Crucially, when we evaluate our classifier on the test set, we are working with a *fixed* classifier (it does not depend on the sample of the test set), and thus estimating its error is simply the problem of mean estimation. However the same cannot be said for the training set. Note that the model we wind up with depends explicitly on the selection of the training set and thus the training error will in general be a biased estimate of the true error on the underlying population. The central question of generalization is then when should we expect our training error to be close to the population error (and thus the generalization error).

Model Complexity

In classical theory, when we have simple models and abundant data, the training and generalization errors tend to be close. However, when we work with more complex models and/or fewer examples, we expect the training error to go down but the generalization gap to grow. This should not be surprising. Imagine a model class so expressive that for any dataset of n examples, we can find a set of parameters that can perfectly fit arbitrary labels, even if randomly assigned. In this case, even if we fit our training data perfectly, how can we conclude anything about the generalization error? For all we know, our generalization error might be no better than random guessing.

In general, absent any restriction on our model class, we cannot conclude, based on fitting the training data alone, that our model has discovered any generalizable pattern (Vapnik *et al.*, 1994). On the other hand, if our model class was not capable of fitting arbitrary labels, then it must have discovered a pattern. Learning-theoretic ideas about model complexity derived some inspiration from the ideas of Karl Popper, an influential philosopher of science, who formalized the criterion of falsifiability. According to Popper, a theory that can explain any and all observations is not a scientific theory at all! After all, what has it told us about the world if it has not ruled out any possibility? In short, what we want is a hypothesis that *could not* explain any observations we might conceivably make and yet nevertheless happens to be compatible with those observations that we *in fact* make.

Now what precisely constitutes an appropriate notion of model complexity is a complex matter. Often, models with more parameters are able to fit a greater number of arbitrarily assigned labels. However, this is not necessarily true. For instance, kernel methods operate in spaces with infinite numbers of parameters, yet their complexity is controlled by other means (Schölkopf and Smola, 2002). One notion of complexity that often proves useful is the range of values that the parameters can take. Here, a model whose parameters are permitted to take arbitrary values would be more complex. We will revisit this idea in the next section, when we introduce *weight decay*, your first practical regularization technique. Notably, it can be difficult to compare complexity among members of substantially different model classes (say, decision trees vs. neural networks).

At this point, we must stress another important point that we will revisit when introducing deep neural networks. When a model is capable of fitting arbitrary labels, low training error does not necessarily imply low generalization error. *However, it does not necessarily*

imply high generalization error either! All we can say with confidence is that low training error alone is not enough to certify low generalization error. Deep neural networks turn out to be just such models: while they generalize well in practice, they are too powerful to allow us to conclude much on the basis of training error alone. In these cases we must rely more heavily on our holdout data to certify generalization after the fact. Error on the holdout data, i.e., validation set, is called the *validation error*.

3.6.2 Underfitting or Overfitting?

When we compare the training and validation errors, we want to be mindful of two common situations. First, we want to watch out for cases when our training error and validation error are both substantial but there is a little gap between them. If the model is unable to reduce the training error, that could mean that our model is too simple (i.e., insufficiently expressive) to capture the pattern that we are trying to model. Moreover, since the *generalization gap* ($R_{\text{emp}} - R$) between our training and generalization errors is small, we have reason to believe that we could get away with a more complex model. This phenomenon is known as *underfitting*.

On the other hand, as we discussed above, we want to watch out for the cases when our training error is significantly lower than our validation error, indicating severe *overfitting*. Note that overfitting is not always a bad thing. In deep learning especially, the best predictive models often perform far better on training data than on holdout data. Ultimately, we usually care about driving the generalization error lower, and only care about the gap insofar as it becomes an obstacle to that end. Note that if the training error is zero, then the generalization gap is precisely equal to the generalization error and we can make progress only by reducing the gap.

Polynomial Curve Fitting

To illustrate some classical intuition about overfitting and model complexity, consider the following: given training data consisting of a single feature x and a corresponding real-valued label y, we try to find the polynomial of degree d

$$\hat{y} = \sum_{i=0}^{d} x^i w_i \tag{3.6.3}$$

for estimating the label y. This is just a linear regression problem where our features are given by the powers of x, the model's weights are given by w_i, and the bias is given by w_0 since $x^0 = 1$ for all x. Since this is just a linear regression problem, we can use the squared error as our loss function.

A higher-order polynomial function is more complex than a lower-order polynomial function, since the higher-order polynomial has more parameters and the model function's selection range is wider. Fixing the training dataset, higher-order polynomial functions should always achieve lower (at worst, equal) training error relative to lower-degree polynomials. In fact, whenever each data example has a distinct value of x, a polynomial function with degree equal to the number of data examples can fit the training set perfectly. We compare

the relationship between polynomial degree (model complexity) and both underfitting and overfitting in Fig. 3.6.1.

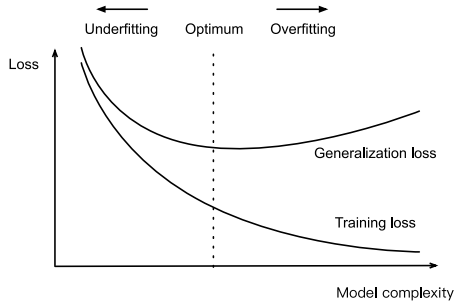


Fig. 3.6.1 Influence of model complexity on underfitting and overfitting.

Dataset Size

As the above bound already indicates, another big consideration to bear in mind is dataset size. Fixing our model, the fewer samples we have in the training dataset, the more likely (and more severely) we are to encounter overfitting. As we increase the amount of training data, the generalization error typically decreases. Moreover, in general, more data never hurts. For a fixed task and data distribution, model complexity should not increase more rapidly than the amount of data. Given more data, we might attempt to fit a more complex model. Absent sufficient data, simpler models may be more difficult to beat. For many tasks, deep learning only outperforms linear models when many thousands of training examples are available. In part, the current success of deep learning owes considerably to the abundance of massive datasets arising from Internet companies, cheap storage, connected devices, and the broad digitization of the economy.

3.6.3 Model Selection

Typically, we select our final model only after evaluating multiple models that differ in various ways (different architectures, training objectives, selected features, data preprocessing, learning rates, etc.). Choosing among many models is aptly called *model selection*.

In principle, we should not touch our test set until after we have chosen all our hyperparameters. Were we to use the test data in the model selection process, there is a risk that we might overfit the test data. Then we would be in serious trouble. If we overfit our training data, there is always the evaluation on test data to keep us honest. But if we overfit the test data, how would we ever know? See Ong *et al.* (2005) for an example of how this can lead to absurd results even for models where the complexity can be tightly controlled.

Thus, we should never rely on the test data for model selection. And yet we cannot rely solely on the training data for model selection either because we cannot estimate the generalization error on the very data that we use to train the model.

In practical applications, the picture gets muddier. While ideally we would only touch the

test data once, to assess the very best model or to compare a small number of models with each other, real-world test data is seldom discarded after just one use. We can seldom afford a new test set for each round of experiments. In fact, recycling benchmark data for decades can have a significant impact on the development of algorithms, e.g., for image classification[81] and optical character recognition[82].

[81] 

The common practice for addressing the problem of *training on the test set* is to split our data three ways, incorporating a *validation set* in addition to the training and test datasets. The result is a murky business where the boundaries between validation and test data are worryingly ambiguous. Unless explicitly stated otherwise, in the experiments in this book we are really working with what should rightly be called training data and validation data, with no true test sets. Therefore, the accuracy reported in each experiment of the book is really the validation accuracy and not a true test set accuracy.

[82] 

Cross-Validation

When training data is scarce, we might not even be able to afford to hold out enough data to constitute a proper validation set. One popular solution to this problem is to employ *K-fold cross-validation*. Here, the original training data is split into K non-overlapping subsets. Then model training and validation are executed K times, each time training on $K - 1$ subsets and validating on a different subset (the one not used for training in that round). Finally, the training and validation errors are estimated by averaging over the results from the K experiments.

3.6.4 Summary

This section explored some of the underpinnings of generalization in machine learning. Some of these ideas become complicated and counterintuitive when we get to deeper models; here, models are capable of overfitting data badly, and the relevant notions of complexity can be both implicit and counterintuitive (e.g., larger architectures with more parameters generalizing better). We leave you with a few rules of thumb:

1. Use validation sets (or *K-fold cross-validation*) for model selection;
2. More complex models often require more data;
3. Relevant notions of complexity include both the number of parameters and the range of values that they are allowed to take;
4. Keeping all else equal, more data almost always leads to better generalization;
5. This entire talk of generalization is all predicated on the IID assumption. If we relax this assumption, allowing for distributions to shift between the train and testing periods, then we cannot say anything about generalization absent a further (perhaps milder) assumption.

3.6.5 Exercises

1. When can you solve the problem of polynomial regression exactly?